🏛️ High Level Design (HLD) — Master Syllabus

From Foundations to Production-Grade Distributed Systems

> **Mentorship Philosophy:** Design is not about memorising patterns — it's about developing intuition. Every module here challenges you to *think*, not just *learn*. Each case study is chosen because it broke something at scale and forced an evolution.

📋 Course Overview

Attribute	Detail
Duration	20 Weeks (Part-time) / 10 Weeks (Intensive)
Level	Beginner → Advanced
Format	Modules → Concepts → Case Studies → Design Exercises
Assessment	Weekly Design Reviews + Capstone System Design
Pre-requisites	Basic programming knowledge, REST API understanding

---

🗺️ Learning Journey Map

```
PHASE 1 (Weeks 1-4)    →    PHASE 2 (Weeks 5-10)    →    PHASE 3 (Weeks 11-16)
→    PHASE 4 (Weeks 17-20)
Foundations                 Core Distributed Systems       Advanced Patterns
Mastery & Capstone
```

PHASE 1 — Foundations of System Design

> *"Before you design a system, understand what a system IS."*

MODULE 1 — What Is High Level Design?

1.1 The Design Mindset

- HLD vs LLD — where they begin and end
- Functional vs Non-functional requirements
- The art of asking the RIGHT clarifying questions
- Understanding the problem space before jumping to solutions
- Stakeholder mapping: who cares about what?

1.2 Core Building Blocks (Mental Models)

- Clients, Servers, Networks — the eternal triangle
- Stateless vs Stateful services
- Synchronous vs Asynchronous communication
- Push vs Pull architectures
- The concept of a "system boundary"

1.3 How to Approach a Design Interview / Design Session

- The 4-step framework: Clarify → Estimate → Design → Deep-dive
- Drawing readable architecture diagrams
- Communicating trade-offs with confidence
- What "good enough" means in system design

🔬 Case Study 1.1 — URL Shortener (bit.ly)

> A deceptively simple system. Students design it, then discover the chaos of 301 vs 302 redirects, hash collisions, analytics requirements, and multi-region replication — all from a "simple" link shortener.

Key Learnings: CRUD at scale, hashing strategies, database choice rationale, caching layer introduction.

MODULE 2 — Networking & Communication Fundamentals

2.1 Protocols That Matter

- HTTP/1.1 vs HTTP/2 vs HTTP/3 — and why it matters at scale
- WebSockets: when polling dies
- gRPC and Protocol Buffers — the RPC renaissance
- GraphQL as an API strategy, not just a query language

- Server-Sent Events (SSE) — the underrated middle ground

2.2 DNS, CDN & Load Balancers

- DNS resolution lifecycle — from browser to IP
- CDN architecture: edge nodes, origin servers, cache invalidation
- Load balancing algorithms: Round Robin, Least Connections, IP Hash, Weighted
- Layer 4 vs Layer 7 load balancing
- Health checks and circuit breakers at the LB level

2.3 API Design Principles

- REST design best practices: versioning, idempotency, HATEOAS
- API Gateway: rate limiting, auth, routing, transformation
- Idempotency keys — why they save distributed systems
- Pagination patterns: offset, cursor, keyset

🔬 Case Study 2.1 — Netflix Video Delivery (CDN Architecture)

> How Netflix moved from a single data centre to a globally distributed CDN (Open Connect). Students redesign the content delivery problem from scratch.

Key Learnings: CDN placement strategy, adaptive bitrate streaming, cache-hit optimisation, origin shield pattern.

MODULE 3 — Storage Systems & Databases

3.1 Relational Databases Deep Dive

- ACID transactions — what each letter *actually* means
- Indexing strategies: B-Tree, Hash, Composite, Partial
- Query planning and the EXPLAIN command
- N+1 problem and how ORMs make it worse
- Replication: Primary-Replica, Multi-Primary

3.2 NoSQL — The Right Tool for the Right Job

- Document stores (MongoDB): schema flexibility trade-offs
- Key-Value stores (Redis, DynamoDB): when you need speed

- Wide-column stores (Cassandra, HBase): time-series and write-heavy workloads
- Graph databases (Neo4j): relationship-first data modelling
- Search engines (Elasticsearch): inverted indexes, relevance scoring

3.3 Database Selection Framework

- The CAP theorem — and why it's often misapplied
- PACELC: a more complete model
- Read-heavy vs Write-heavy workload patterns
- The "Polyglot Persistence" architecture

3.4 Data Modelling at Scale

- Denormalisation as a deliberate strategy
- Partitioning vs Sharding — the difference matters
- Consistent hashing for shard routing
- Hot partition problem and how to solve it

🔬 Case Study 3.1 — Instagram's Database Evolution

> Instagram started on PostgreSQL alone. Students trace the journey from a single Postgres instance → read replicas → Cassandra for feeds → Redis for counters → Elasticsearch for search.

Key Learnings: Evolutionary database architecture, the cost of migrations, matching storage to access pattern.

MODULE 4 — Caching — The Great Accelerator

4.1 Cache Fundamentals

- Why caching works: temporal and spatial locality
- Cache hit rate and its impact on system performance
- Cache levels: L1/L2/L3 CPU → Application → Distributed → CDN

4.2 Caching Strategies

- Cache-Aside (Lazy Loading) — the most common pattern
- Write-Through — consistency over performance

- Write-Behind (Write-Back) — performance over consistency
- Read-Through — transparent to the application
- Refresh-Ahead — predicting access patterns

4.3 Cache Invalidation — "The Hardest Problem"

- TTL-based invalidation and its pitfalls
- Event-driven invalidation
- Cache stampede / thundering herd problem
- Stale-While-Revalidate pattern
- Probabilistic early expiration

4.4 Redis Architecture

- Redis data structures and when to use each
- Redis Pub/Sub vs Streams
- Redis Cluster and consistent hashing
- Sentinel vs Cluster for HA

🔬 Case Study 4.1 — Twitter's Home Timeline Cache

> Twitter's timeline service reads 300K tweets per second at peak. Students design a fan-out-on-write vs fan-out-on-read system, and discover why Twitter uses a hybrid for celebrities (the "Lady Gaga problem").

Key Learnings: Fan-out strategies, cache warming, handling celebrity users as edge cases, Redis sorted sets for feed ranking.

PHASE 2 — Core Distributed Systems

> *"Scale breaks everything you designed in Phase 1. Now you learn to design with that in mind."*

MODULE 5 — Scalability Patterns

5.1 Horizontal vs Vertical Scaling

- Vertical scaling limits — Moore's Law is slowing
- Stateless service design as a prerequisite for horizontal scale

- Session management in horizontally scaled systems
- Shared-nothing architecture

5.2 Sharding at Scale

- Functional sharding (sharding by domain/service)
- Horizontal sharding strategies: Range, Hash, Directory-based
- Cross-shard queries — the unavoidable pain
- Resharding without downtime

5.3 Rate Limiting & Throttling

- Token Bucket algorithm — the most popular choice and why
- Leaky Bucket — smoothing bursty traffic
- Fixed Window Counter — simple but flawed
- Sliding Window Log — accurate but memory-heavy
- Sliding Window Counter — the practical sweet spot
- Distributed rate limiting with Redis

5.4 Backpressure & Bulkheads

- Backpressure: letting upstream know you're overwhelmed
- Bulkhead pattern: isolating failure domains
- Shed load gracefully vs fail silently
- Adaptive throttling in gRPC

🔬 Case Study 5.1 — Uber's Surge Pricing Engine

> Uber computes surge pricing every minute across 10,000+ cities. Students design the geospatial aggregation pipeline, rate updates, and driver supply/demand matching system.

Key Learnings: Geospatial indexing (S2 cells), time-windowed aggregation, consistency requirements in pricing, event-driven architecture.

MODULE 6 — Message Queues & Event-Driven Architecture

6.1 Why Asynchrony Matters

- Decoupling producers from consumers

- Handling traffic spikes with queues as buffers
- The concept of eventual consistency as a feature
- At-most-once vs At-least-once vs Exactly-once delivery

6.2 Message Queue Deep Dive

- RabbitMQ: exchanges, bindings, routing keys — AMQP model
- Apache Kafka: partitions, consumer groups, offsets, compaction
- SQS/SNS: managed simplicity and its trade-offs
- Kafka vs RabbitMQ — choosing the right tool

6.3 Event-Driven Architecture Patterns

- Event Notification vs Event-Carried State Transfer vs Event Sourcing
- Saga Pattern: managing distributed transactions without 2PC
- Outbox Pattern: guaranteed message delivery from databases
- Dead Letter Queues (DLQ) and poison message handling
- Event schema evolution and backward compatibility

6.4 CQRS — Command Query Responsibility Segregation

- Separating read and write models
- Projections and read model rebuilding
- CQRS + Event Sourcing: a powerful but complex combination
- When NOT to use CQRS

🔬 Case Study 6.1 — Airbnb's Messaging Platform Migration

> Airbnb migrated from a monolithic messaging system to an event-driven platform using Kafka. Students re-architect the host-guest messaging, booking events, and notification fan-out.

Key Learnings: Event ordering guarantees, idempotent consumers, schema registry, real-world Kafka partition strategy.

7.1 The Challenges of Distribution

- Network partitions — not a theoretical concern
- Clock skew and the impossibility of global time
- The Two Generals Problem
- Byzantine fault tolerance — what it means in practice

7.2 Consistency Models

- Strong consistency — the gold standard with a price
- Eventual consistency — the scalable trade-off
- Causal consistency — the sweet spot
- Read-your-writes, Monotonic reads, Monotonic writes
- Linearizability vs Serializability — a critical distinction

7.3 Consensus Algorithms

- The need for leader election
- Paxos — why it's hard to understand and implement
- Raft — designed for understandability
- How etcd and ZooKeeper use consensus
- Leader election with Redis (Redlock controversy)

7.4 Distributed Transactions

- Two-Phase Commit (2PC) — the classic with its failure modes
- Three-Phase Commit (3PC)
- Saga pattern as a 2PC alternative
- Compensating transactions

🔬 Case Study 7.1 — Google Spanner

> Spanner achieves global strong consistency using TrueTime (GPS + atomic clocks). Students analyse why Spanner's design is theoretically impossible by CAP, and understand how Google sidestpped it.

Key Learnings: TrueTime API, external consistency, globally distributed SQL, the cost of global coordination.

MODULE 8 — Search Systems

8.1 Search Fundamentals

- Inverted index — the data structure that powers search
- Tokenisation, stemming, lemmatisation
- TF-IDF scoring
- BM25 — the modern relevance standard

8.2 Elasticsearch Architecture

- Index, Shard, Replica — the storage model
- Segment merging and near-real-time indexing
- Query DSL: bool queries, filters, aggregations
- Relevance tuning: boosting, function score, field weighting

8.3 Typeahead & Autocomplete

- Trie data structure for prefix search
- Fuzzy matching with Levenshtein distance
- Phonetic matching (Soundex, Metaphone)
- Building a scalable typeahead with Redis sorted sets

8.4 Vector Search & Semantic Search

- Embeddings and vector representations
- Approximate Nearest Neighbour (ANN): HNSW, IVF
- Hybrid search: combining keyword + vector
- Pgvector, Pinecone, Weaviate overview

🔬 Case Study 8.1 — LinkedIn's People Search

> LinkedIn serves 900M+ members with a search system that ranks by relevance AND network distance. Students design the ingestion pipeline, ranking model inputs, and real-time index updates.

Key Learnings: Multi-factor ranking, search relevance as a product problem, near-real-time index updates, A/B testing relevance.

PHASE 3 — Advanced Distributed Patterns

> *"This is where senior engineers separate themselves. You're not just designing — you're anticipating failure."*

MODULE 9 — Microservices Architecture

9.1 The Microservices Philosophy

- Conway's Law — organisational structure mirrors architecture
- Domain-Driven Design (DDD): Bounded Contexts, Aggregates, Ubiquitous Language
- Service decomposition strategies: by domain, by team, by volatility
- The 12-Factor App methodology

9.2 Inter-Service Communication

- Synchronous: REST, gRPC — trade-offs at scale
- Asynchronous: Events, Messages — decoupling benefits
- Service Mesh: Istio, Linkerd — sidecar proxy model
- Service discovery: client-side vs server-side

9.3 Failure Handling in Microservices

- Circuit Breaker pattern: Closed, Open, Half-Open states
- Retry with exponential backoff and jitter
- Timeout configuration — the "right" timeout doesn't exist
- Fallback strategies: cached data, degraded responses, feature flags
- Chaos Engineering: intentionally breaking things in production

9.4 Distributed Observability

- The three pillars: Logs, Metrics, Traces
- Distributed tracing with OpenTelemetry
- Correlation IDs across service boundaries
- SLI, SLO, SLA — defining and measuring reliability
- Error budgets and their use in engineering decisions

> Amazon famously mandated in 2002 that every team must communicate via APIs only. Students analyse the "Bezos API Mandate" and redesign a hypothetical e-commerce monolith into services.

Key Learnings: The real cost of microservices, organisational coupling vs technical coupling, when microservices make things worse.

MODULE 10 — Micro Frontend Architecture

> *"Microservices for the browser — distributed UI ownership at scale."*

10.1 What Is Micro Frontend Architecture?

- The monolithic frontend problem at scale
- Independent deployability for UI teams
- Micro Frontends as an organisational pattern, not just technical
- When micro frontends are the WRONG choice

10.2 Integration Patterns

10.2.1 Build-Time Integration

- NPM packages as shared UI components
- Benefits: type safety, tree shaking
- Drawbacks: deployment coupling, version hell
- Mono-repo strategies with Nx and Turborepo

10.2.2 Server-Side Composition

- Edge-Side Includes (ESI)
- Server-Side Includes (SSI)
- Zalando's Tailor / Mosaic architecture
- Trade-offs: SEO, performance, complexity

10.2.3 Runtime Integration via iFrames

- True isolation — security and style sandboxing
- The communication problem: postMessage API

- When iFrames are actually the right answer (fintech, embedded widgets)

10.2.4 Runtime Integration via JavaScript

- Module Federation (Webpack 5) — the paradigm shift
- Dynamic remote loading at runtime
- Shared dependencies: the version negotiation problem
- Vite Module Federation plugin

10.2.5 Web Components

- Custom Elements, Shadow DOM, HTML Templates
- Framework-agnostic UI components
- Trade-offs with React ecosystem

10.3 Module Federation — Deep Dive

- Host vs Remote architecture
- Eager vs Lazy loading of remotes
- Shared module configuration and singleton enforcement
- Federated types for TypeScript projects
- Rollback strategy for federated modules

10.4 Routing in Micro Frontends

- Shell application as the orchestrator
- History-based routing vs Hash routing in composed apps
- Deep linking across federated apps
- React Router v6 in federated contexts

10.5 Shared State & Communication

- Props / Custom Events for parent-child communication
- Shared state store: Redux, Zustand across federation boundaries
- Event Bus pattern for cross-MFE communication
- URL as shared state — the simplest solution

10.6 Shared Design System

- Design tokens as the foundation
- Component library as a federated remote
- Versioning strategy for shared components
- Storybook for federated component documentation

10.7 Authentication & Security

- Single Sign-On (SSO) in micro frontend context
- JWT token sharing across apps
- Cookie-based auth vs token-based in composed shells
- CSP headers with federated scripts

10.8 Performance in Micro Frontends

- Bundle size governance per team
- Lazy loading micro frontends on route change
- Critical CSS in composed pages
- Performance budgets per MFE

10.9 Testing Strategies

- Unit testing within bounded context
- Contract testing with Pact
- Integration testing the shell + remotes
- E2E testing across micro frontend boundaries

🔬 Case Study 10.1 — IKEA's Micro Frontend Migration

> IKEA migrated their e-commerce frontend (serving 4.3B visits/year) from a monolith to micro frontends. Students re-design the product listing page, checkout flow, and account section as independent federated apps.

Key Learnings: Team topology to MFE mapping, shared cart state across MFEs, SEO implications, performance regression detection.

🔬 Case Study 10.2 — Spotify's "Backstage" Platform

> Spotify built an internal developer portal as a micro frontend system. Students design the plugin architecture, shared navigation, and how 150+ teams contribute independently.

Key Learnings: Internal developer platforms, plugin architecture, governance without gatekeeping, federated documentation.

🔬 Case Study 10.3 — Large-Scale E-Commerce MFE (Full Design Exercise)

> Design a complete e-commerce platform (think Flipkart/Amazon India scale) using micro frontends:
>
> - **Shell App**: Navigation, auth, routing orchestration
> - **Product MFE**: Catalogue, PDP, search integration
> - **Cart & Checkout MFE**: Cart state, payment integration
> - **User Account MFE**: Profile, orders, addresses
> - **Recommendations MFE**: ML-powered widgets embedded across pages

Students produce architecture diagrams, team ownership mapping, deployment strategy, and performance SLAs per MFE.

MODULE 11 — Data-Intensive System Design

11.1 Stream Processing

- Batch vs Stream processing — the fundamental difference
- Apache Kafka Streams vs Apache Flink vs Spark Streaming
- Watermarks and handling late-arriving events
- Stateful stream processing: aggregations, joins, windowing

11.2 Data Pipeline Architecture

- Lambda Architecture: batch + speed layer
- Kappa Architecture: stream-only simplicity
- Change Data Capture (CDC) with Debezium
- ETL vs ELT — the shift to cloud data warehouses

11.3 Analytical Systems

- OLTP vs OLAP — a fundamental distinction
- Column-oriented storage and its advantages
- Data warehouse vs Data lake vs Data lakehouse
- Partitioning and clustering in BigQuery / Snowflake

🔬 Case Study 11.1 — Uber's Real-Time Analytics (Pinot)

> Uber needed sub-second analytics on 100M+ ride events per day for their operational dashboards. Students design the ingestion pipeline, storage optimisation, and query execution.

Key Learnings: Apache Pinot's architecture, upsert handling in analytical stores, indexing for analytical queries, real-time vs historical query paths.

MODULE 12 — Notification & Communication Systems

12.1 Notification System Architecture

- Multi-channel delivery: Push, Email, SMS, In-App
- Notification preferences and user-level opt-outs
- Priority queues for critical vs marketing notifications
- Deduplication and idempotency in notification delivery

12.2 Real-Time Communication

- WebSocket connection management at scale
- Long polling fallback strategy
- Presence systems: online/offline/typing indicators
- Message ordering in chat systems

12.3 Email Delivery at Scale

- SMTP vs SES vs SendGrid — when managed services win
- Bounce handling, complaint handling, unsubscribes
- Email deliverability: SPF, DKIM, DMARC
- Transactional vs bulk email separation

🔬 Case Study 12.1 — WhatsApp Architecture

> WhatsApp serves 100B+ messages/day with 2-person team per server. Students design the message storage, delivery receipt system, end-to-end encryption key distribution, and presence system.

Key Learnings: Erlang's actor model, message delivery guarantees, offline message queuing, the cost of receipts at scale.

PHASE 4 — Mastery, Reliability & Capstone

> *"Senior engineers obsess over failure modes. Great architects design for the failure they haven't imagined yet."*

MODULE 13 — Reliability Engineering

13.1 Designing for Failure

- Everything fails — designing with that assumption
- Mean Time Between Failures (MTBF) vs Mean Time to Recovery (MTTR)
- The difference between fault tolerance and high availability
- Five 9s (99.999%) — what it actually costs

13.2 Replication & Failover

- Active-Passive vs Active-Active replication
- Automatic failover: detection, election, promotion
- Split-brain problem and fencing tokens
- Geo-redundancy and disaster recovery

13.3 Chaos Engineering

- Netflix Simian Army: Chaos Monkey, Latency Monkey, Conformity Monkey
- Game Days: simulating failures intentionally
- Blast radius: limiting the scope of experiments
- Building a culture of chaos in cautious organisations

13.4 Graceful Degradation

- Feature flags for progressive rollout and emergency kill switches
- Returning stale data > returning errors
- Serving cached responses during database outages
- Partial failures and partial responses

🔬 **Case Study 13.1 — GitHub's Database Failover Incident (2018)**

> A 43-second network partition caused GitHub's MySQL replication to fall behind by 166 seconds, leading to a 24-hour partial outage. Students analyse the failure chain and design mitigations.

Key Learnings: The danger of automation in partial failure, human decision-making in incidents, post-mortem culture.

MODULE 14 — Security in System Design

14.1 Authentication & Authorisation

- OAuth 2.0 flows: Authorization Code, Client Credentials, Device Flow
- JWT: structure, signing, validation, refresh strategy
- RBAC vs ABAC vs ReBAC (relationship-based access control)
- API key management and rotation

14.2 Common Attack Vectors at Scale

- DDoS: mitigation strategies at CDN, LB, and application layers
- SQL Injection: parameterised queries and ORMs
- SSRF (Server-Side Request Forgery): metadata endpoint attacks
- Mass assignment and insecure direct object references

14.3 Data Privacy & Compliance

- PII handling: identification, classification, minimisation
- GDPR right-to-erasure: deletion in distributed systems
- Data residency requirements in multi-region systems
- Encryption at rest and in transit — choosing the right strategy

---

MODULE 15 — Cost-Aware System Design

15.1 Cloud Cost Architecture

- Compute: Reserved vs On-Demand vs Spot instances
- Storage: Hot vs Cool vs Archive tiers

- Network egress costs — the "hidden" bill
- Cost per request as a design metric

15.2 Optimisation Patterns

- Request coalescing: fewer calls, same data
- Compression: gzip, Brotli, Zstandard trade-offs
- Binary protocols over JSON at scale
- Tiered storage: keeping hot data fast, cold data cheap

MODULE 16 — Capstone System Designs

> Students are given 72 hours to produce a complete design document for one of the following systems. Design is then reviewed in a mentor-led session.

Capstone Option A — Design YouTube at Scale

Full video upload pipeline, transcoding, CDN delivery, recommendation engine, search, real-time view counter, and comment system. Design for 500 hours of video uploaded per minute.

Capstone Option B — Design a Ride-Sharing Platform (India Scale)

Driver matching, real-time location tracking (100K concurrent drivers), surge pricing, ETA computation, payment processing, and driver earnings settlement.

Capstone Option C — Design a Live Collaboration Tool (Figma/Google Docs)

Operational Transformation vs CRDT for conflict resolution, real-time cursor presence, version history, commenting, and offline-first sync.

Capstone Option D — Design an E-Commerce Platform with Micro Frontends

Full-stack design covering: micro frontend architecture, product catalogue, inventory management, order processing, payment gateway integration, and real-time delivery tracking.

⚙️ Famous Techniques in System Design

> *"These are the tools every senior engineer reaches for. Know them deeply — not just their names."*

Problem it solves: When you add or remove a node in a distributed cache or database cluster, naive modulo hashing (`key % N`) remaps almost ALL keys, causing a thundering herd on the new nodes.

How it works:

- Arrange nodes on a virtual ring of 2^{32} positions (hash space)
- Each node is assigned a position via `hash(node_id)`
- A key is routed to the **first node clockwise** from `hash(key)`
- When a node is added/removed, only keys between its predecessor and itself are remapped — typically `1/N` of total keys

Virtual Nodes (VNodes):

- Real nodes get multiple positions on the ring (e.g., 150 vnodes per node)
- Ensures even load distribution even with heterogeneous hardware
- Amazon DynamoDB, Apache Cassandra, and Riak all use this

Used by: Amazon DynamoDB, Apache Cassandra, Memcached (ketama), Nginx upstream hashing

When to use it:

- Distributed caching with dynamic node membership
- Partitioning data across shards where nodes join/leave frequently

Naive Hashing	Consistent Hashing
~100% keys remapped on node change	~1/N keys remapped
Simple to implement	Slightly more complex
Works for static clusters	Works for dynamic clusters

TECHNIQUE 2 — Bloom Filters

Problem it solves: "Is this item definitely NOT in this set?" — answering this without loading the entire dataset into memory.

How it works:

- A bit array of `m` bits, all initialised to 0
- `k` independent hash functions
- **Insert**: hash the item `k` times, set those bit positions to 1
- **Query**: hash the item `k` times — if ALL positions are 1, item *may* exist; if ANY position is 0, item *definitely does not exist*
- **No false negatives. False positives are possible.**

Trade-off formula: False positive rate ≈ `(1 - e^(-kn/m))^k`

Real-world applications:

- **Chrome Safe Browsing**: checks URLs against a bloom filter of malicious sites locally before hitting a server
- **Cassandra**: checks bloom filter before reading SSTable files from disk (saves I/O)
- **Medium**: prevents showing already-read articles
- **Akamai CDN**: filters one-hit wonders from being cached at the edge
- **HBase / RocksDB**: reduces unnecessary disk lookups

Variants:

- **Counting Bloom Filter**: supports deletions (counts instead of bits)
- **Cuckoo Filter**: supports deletion with better performance
- **XOR Filter**: more space-efficient, read-only

TECHNIQUE 3 — HyperLogLog (HLL)

Problem it solves: Counting unique items (cardinality) in a massive stream without storing each item — e.g., "How many unique users visited today?"

How it works:

- Uses probabilistic counting with hash functions
- Observes the maximum number of leading zeros in hashed values
- Maintains only `O(log log N)` memory regardless of stream size
- Standard error: ~1.04 / √m where m is the number of registers

Real numbers:

- Count billions of unique items with **~1.5% error**
- Uses only **~12 KB of memory** (vs gigabytes for exact counting)

Used by:

- **Redis**: `PFADD` / `PFCOUNT` commands are HLL natively
- **Google BigQuery**: `APPROX_COUNT_DISTINCT()`
- **Apache Spark**: built-in HLL support
- **PostreSQL**: `pg_hll` extension

When to use it: Real-time analytics dashboards, unique visitor counts, A/B test unique exposure counts, any cardinality estimate where ~1–2% error is acceptable.

TECHNIQUE 4 — Count-Min Sketch

Problem it solves: "What are the approximate frequencies of items in a data stream?" — without storing every item. A frequency table that fits in memory.

How it works:

- A 2D array of `d` rows × `w` columns, all zeros
- `d` hash functions, one per row
- **Increment**: for item `x`, increment `table[i][hash_i(x)]` for each row `i`
- **Query**: return the MINIMUM across all rows for item `x` (overestimates, never underestimates)

Used by:

- **Apache Storm**: heavy-hitter detection in stream processing
- **Twitter**: tracking trending hashtags and heavy hitters in real time
- **Redis**: used internally in Redis-CRDTs
- **Network traffic analysis**: detecting DDoS source IPs

Related: Heavy Hitter algorithms (finding the top-K most frequent items) use Count-Min Sketch as a subroutine.

Problem it solves: Efficiently verifying that two large datasets are identical — and pinpointing exactly WHERE they differ, without transferring the entire dataset.

How it works:

- Leaf nodes contain hashes of individual data blocks
- Internal nodes contain the hash of their children's hashes
- Root hash = fingerprint of the entire dataset
- To compare two trees: compare root hashes. If different, recurse into children to find the divergent leaf

Used by:

- **Bitcoin / Ethereum**: transaction integrity in blocks, efficient SPV proof
- **Amazon DynamoDB / Cassandra**: anti-entropy repair — detecting which vnodes have diverged between replicas
- **Git**: the entire object model is a Merkle DAG
- **AWS Certificate Transparency**: certificate log verification
- **IPFS / BitTorrent**: chunk-level integrity verification

Why it matters in system design: In a distributed database with 10 replicas, instead of comparing 100GB of data to detect divergence, you compare a single 32-byte hash at the root, then drill down to find the specific mismatched chunks.

TECHNIQUE 6 — Consistent Snapshots (MVCC)

Problem it solves: Allowing concurrent reads and writes without locking — readers never block writers, writers never block readers.

How it works (Multi-Version Concurrency Control):

- Every write creates a NEW version of a row, stamped with a transaction ID (TXID)
- Reads see a consistent snapshot of all data committed *before their transaction started*
- Old versions are cleaned up by a background `VACUUM` process
- No read locks required — each transaction sees its own consistent view

Used by:

- **PostgreSQL**: heap-based MVCC
- **MySQL InnoDB**: undo log-based MVCC
- **CockroachDB / Spanner**: MVCC with hybrid logical clocks for distributed snapshots
- **FoundationDB**: strict serializability via MVCC + OCC

Trade-offs:

- Bloat: old versions accumulate and must be garbage collected
- Write amplification: every update is an insert + a tombstone on the old version
- Read consistency: long-running transactions can see very stale data

---

TECHNIQUE 7 — Leaderless Replication & Quorum Reads/Writes

Problem it solves: Eliminating the single leader as a bottleneck and single point of failure in replicated systems.

How it works:

- All replicas accept reads and writes directly (no designated leader)
- A write is sent to `W` replicas simultaneously; success if `W` acknowledge
- A read is sent to `R` replicas simultaneously; return the most recent version
- **Quorum condition**: `W + R > N` guarantees at least one node has the latest write
  - Typical: N=3, W=2, R=2

Read Repair: When a read detects stale replicas (via version vectors), it writes the latest value back to those replicas.

Anti-entropy: Background process continuously syncs replicas using Merkle tree comparison.

Used by:

- **Amazon DynamoDB**: eventual consistency with tunable quorum
- **Apache Cassandra**: `CONSISTENCY QUORUM` / `ONE` / `ALL` per query
- **Riak**: Dynamo-inspired leaderless replication

Trade-offs:

- Edge cases: sloppy quorum, hinted handoff during node failures
- Last-Write-Wins (LWW) conflict resolution can silently discard writes

- CRDTs as a principled alternative to LWW

TECHNIQUE 8 — Rate Limiting Algorithms (Complete Taxonomy)

Token Bucket

- A bucket holds up to `C` tokens; `R` tokens are added per second
- Each request consumes one token; denied if bucket is empty
- **Allows bursting** up to `C` requests instantaneously
- Used by: AWS API Gateway, Stripe

Leaky Bucket

- Requests enter a queue; they drip out at a fixed rate
- Excess requests are dropped (or queued if the bucket isn't full)
- **Smooths traffic** — no bursting allowed
- Used by: NGINX `limit_req_zone`

Fixed Window Counter

- Count requests in a fixed time window (e.g., 100 req/min)
- Simple but has **boundary spike** problem: 200 requests in 2 seconds across a window boundary

Sliding Window Log

- Track timestamp of every request in a sorted set
- On each request, remove timestamps older than window, count remainder
- **Accurate** but memory-heavy (O(max_requests))

Sliding Window Counter

- Hybrid: combine current window count + weighted previous window count
- `allowed = current_count + prev_count × (1 - elapsed_fraction)`
- **Practical and memory-efficient** — used by Cloudflare

TECHNIQUE 9 — Geospatial Indexing Techniques

Geohash

- Encode latitude/longitude as a Base32 string
- Hierarchical: shorter prefix = larger area (e.g., `ttmx` ≈ 25km × 25km)
- Proximity search: query the cell + 8 neighbours
- **Problem**: edge distortion near poles; boundary cells aren't always adjacent

S2 Geometry (Google)

- Maps Earth's surface onto a cube, then subdivides using a Hilbert space-filling curve
- Levels 0–30 (level 30 ≈ $1cm^2$)
- Nearby points remain nearby in S2 index — better than Geohash
- Used by: **Uber, Lyft** (driver matching), **Google Maps**, **Foursquare**

Quadtree

- Recursively divide a 2D space into 4 quadrants
- Each node stores points until a threshold, then splits
- Used for: **proximity search**, **map tile rendering**, collision detection
- Used by: Twitter's geo search, older Foursquare versions

R-Tree

- Used by PostgreSQL/PostGIS for spatial indexing
- Groups nearby objects in minimum bounding rectangles
- Efficient for: range queries, polygon intersection, nearest-neighbour

---

TECHNIQUE 10 — Distributed Locking Patterns

Problem it solves: Mutual exclusion in distributed systems — ensuring only one process executes a critical section at a time.

Redis SETNX (Simple Lock)

```
SET lock:resource_id <unique_token> NX PX 30000
```

- `NX`: only set if not exists
- `PX 30000`: expire in 30 seconds (auto-release on crash)
- Release: compare token before DEL (prevents releasing another process's lock)

Redlock Algorithm (Martin Kleppmann controversy)

- Acquire locks on **N/2+1** independent Redis nodes simultaneously
- Lock is valid only if acquired on the majority within the validity period
- **Controversy**: Martin Kleppmann argues Redlock is unsafe with clock skew; Antirez disagrees
- **Lesson**: Distributed locks are hard. Prefer fencing tokens when possible.

Fencing Tokens (Safe Locking)

- Each lock acquisition returns a monotonically increasing token
- The resource (storage system) rejects requests with older tokens
- Even if a lock holder is paused (GC, network) and its lock expires, its stale write is rejected

ZooKeeper / etcd-based Locks

- Use leader election primitives built on linearisable consensus (Raft/ZAB)
- Ephemeral nodes auto-delete when client disconnects
- Most correct but higher latency than Redis

TECHNIQUE 11 — Long Polling, SSE & WebSocket Comparison

Technique	Connection	Direction	Best For
Short Polling	New per request	Client → Server	Simple status checks
Long Polling	Held open until data	Server → Client (1 msg)	Notifications, chat fallback
Server-Sent Events (SSE)	Persistent (HTTP)	Server → Client (stream)	Dashboards, live feeds, notifications
WebSocket	Persistent (TCP)	Bidirectional	Chat, collaborative editing, games

Technique	Connection	Direction	Best For
gRPC Streaming	Persistent (HTTP/2)	Bidirectional	Service-to-service real-time

SSE is underrated: Works over HTTP/1.1, automatic reconnect built into the browser, works through most proxies, and is perfect for 80% of real-time use cases.

TECHNIQUE 12 — Event Sourcing & CQRS

Event Sourcing:

- Store every state change as an immutable **event**, not the current state
- Current state = replay of all events from the beginning
- Complete audit log for free; temporal queries ("what was the state on Tuesday?")
- **Used by**: Kafka as an event log, bank ledger systems, e-commerce order history

CQRS (Command Query Responsibility Segregation):

- **Commands** mutate state (write side); **Queries** read state (read side)
- Write side: normalised, append-only event store
- Read side: denormalised projections optimised for query patterns
- **Trade-off**: eventual consistency between write and read models; projection rebuilds during schema changes

When NOT to use Event Sourcing:

- Simple CRUD apps — overkill
- Teams without deep DDD understanding — will be misused
- Systems requiring synchronous state reads immediately after writes

---

TECHNIQUE 13 — Back-of-the-Envelope Estimation

> *"Numbers without intuition are just noise. Develop the instinct."*

The Essential Numbers Every Architect Knows

Operation	Latency
L1 cache reference	0.5 ns
L2 cache reference	7 ns
RAM access	100 ns
SSD random read	100 µs
HDD seek	10 ms
Network RTT (same DC)	500 µs
Network RTT (cross-continent)	150 ms
Read 1MB from SSD	1 ms
Read 1MB from network	10 ms

Estimation Template

```
Daily Active Users (DAU):          _______
Requests per user per day:         _______
Total requests/day:                DAU × req/user
Requests per second (RPS):         total / 86,400 ≈ total / 100,000
Peak RPS (assume 3× average):      _______
Storage per record:                _______
New records per day:               _______
Storage per year:                  records/day × 365 × size
```

Quick Rules

- `1M req/day ≈ 12 req/sec`
- `1 Billion req/day ≈ 12,000 req/sec`
- `1 KB × 1M users = 1 GB`
- `1 MB × 1M users = 1 TB`
- Peak traffic is usually **2–3× average**

TECHNIQUE 14 — The Saga Pattern for Distributed Transactions

Problem it solves: Multi-step business transactions that span multiple services — where 2PC is too slow and locks too much.

Choreography-based Saga

- Each service publishes events; downstream services react
- No central coordinator
- ✅ Loose coupling ❌ Hard to track overall transaction state

```
OrderService ──[OrderCreated]──▶ PaymentService ──[PaymentCompleted]──▶
InventoryService ──[ItemReserved]──▶ ShippingService
                                          |
                                   [PaymentFailed]
                                          |
                              ◀──[CancelOrder]── OrderService
```

Orchestration-based Saga

- A central Saga Orchestrator tells each service what to do and handles failures
- ✅ Easier to reason about ❌ Orchestrator can become a bottleneck / single point of failure

Compensating Transactions: Each step must have a defined rollback action:

Step	Action	Compensating Action
Reserve inventory	`reserveItem()`	`releaseItem()`
Charge payment	`chargeCard()`	`refundCard()`
Create shipment	`createShipment()`	`cancelShipment()`

Used by: Uber (trip lifecycle), Amazon (order processing), Netflix (content licensing)

📚 Reference Architecture Cheat Sheets

The HLD Decision Framework

```
REQUIREMENT ANALYSIS
```

```
        │
        ├── Read-heavy? ──────────────► Cache aggressively, Read replicas
        ├── Write-heavy? ─────────────► Queue writes, Async processing, LSM-tree
stores
        ├── Consistency critical? ────► PostgreSQL + Synchronous replication
        ├── Availability critical? ───► NoSQL + Eventual consistency
        ├── Search required? ─────────► Elasticsearch / Typesense
        ├── Real-time required? ──────► WebSockets / Kafka / Redis Pub-Sub
        └── Analytics required? ──────► Columnar store, Separate OLAP system
```

Technology Selection Matrix

Need	Choice	Why
Relational store	PostgreSQL	Mature, extensions, JSONB, FTS
Document store	MongoDB	Flexible schema, aggregation pipeline
KV / Cache	Redis	Data structures, Pub/Sub, Streams
Wide-column	Cassandra	Write-optimised, geo-distribution
Message queue	Kafka	High-throughput, replay, partitioning
Task queue	RabbitMQ	Routing, priorities, dead-lettering
Search	Elasticsearch	Full-text + analytics
Object storage	S3 / GCS	Unlimited scale, 11 9s durability
CDN	Cloudflare / CloudFront	Edge caching, DDoS, WAF
Container orchestration	Kubernetes	Self-healing, auto-scaling, service mesh
Micro Frontends	Module Federation	Runtime integration, independent deploy

🎓 Mentor Notes

What Separates Good from Great Designers

Good Designer	Great Designer
Knows the patterns	Knows when patterns are wrong
Designs for requirements	Designs for change
Thinks about performance	Thinks about failure
Uses popular tools	Questions tool choices
Delivers on time	Delivers with documented trade-offs
Learns from docs	Learns from production incidents

The 10 Questions Great Architects Always Ask

1. **What happens when this component fails?**
2. **What's the read/write ratio and how does it change under load?**
3. **What's the blast radius of a deployment mistake here?**
4. **How does this behave 10x current load?**
5. **How does a new engineer understand this in 6 months?**
6. **What's the cost at scale — per request, per user?**
7. **Where is the single point of failure hiding?**
8. **What's the data consistency requirement — actually?**
9. **How do we monitor that this is working correctly?**
10. **What would I do differently if I knew then what I know now?**

📖 Recommended Reading

Books (In Order of Priority)

1. *Designing Data-Intensive Applications* — Martin Kleppmann *(The Bible)*
2. *System Design Interview Vol. 1 & 2* — Alex Xu
3. *Building Microservices* — Sam Newman

4. *Release It!* — Michael Nygard *(Patterns for production)*
5. *Domain-Driven Design* — Eric Evans *(For service boundaries)*
6. *The Phoenix Project* — Gene Kim *(DevOps culture)*

Engineering Blogs (Must-Follow)

- **Highscalability.com** — Real-world architecture breakdowns
- **Netflix Tech Blog** — Distributed systems at brutal scale
- **Uber Engineering** — Geospatial, real-time systems
- **Martin Fowler's Blog** — Patterns, DDD, microservices
- **AWS Architecture Blog** — Cloud-native patterns
- **Bytebytego** — Visual system design

Syllabus Version 3.0 | Maintained by: HLD Mentorship Programme Last Updated: 2026 | Review Cycle: Every 6 Months